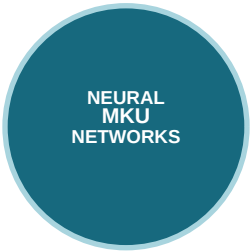
MUSTAFA KEMAL UNIVERSITY
Faculty of Engineering | Department of Computer Science
CSE 589 — Neural Networks



Fake News Detection

Using Deep Learning

A Project Report for CSE 589 — Neural Networks

Submitted by

Abdullah Mohammed Sindi

Student ID: 200044933

Course	Subject	Date
CSE 589	**Neural Networks**	**August 2026**

TensorFlow · Keras · FastAPI · React · Python 3.10

Table of Contents

Abstract

1. Introduction

1.1 Problem Statement

1.2 Why Deep Learning?

1.3 Project Objectives

2. Background

3. Dataset

3.1 Data Collection and Labeling

3.2 Exploratory Analysis

3.3 Engineered Features

4. Methodology

4.1 Text Preprocessing Pipeline

4.2 Feature Extraction — TF-IDF

5. Neural Network Architecture

5.1 Layer Design and Reasoning

5.2 Training Configuration

6. Experiments

6.1 Experiment 1 — Perceptron Baseline

6.2 Experiment 2 — Activation Functions

6.3 Experiment 3 — Optimizer Comparison

6.4 Experiment 4 — Regularization

7. Final Model Results

7.1 Classification Report

7.2 Confusion Matrix Analysis

8. 1D CNN for Text Classification

9. System Architecture and Deployment

10. Challenges Faced

11. Conclusion

References

Abstract

The spread of fake news through social media has emerged as a genuine threat to informed decision-making, political discourse, and public trust in journalism. This report documents a complete deep learning system built to classify news articles as either Fake or Factual, developed as the course project for CSE 589 — Neural Networks at Mustafa Kemal University.

The system is built around a **Feedforward Neural Network** implemented with TensorFlow and Keras, using TF-IDF (Term Frequency-Inverse Document Frequency) vectors as input features. The model was trained on 198 labeled news articles and achieves **97.5% accuracy on the held-out test set with zero false positives** — meaning it never incorrectly flags a genuine article as fake. Four controlled experiments compare activation functions, optimizers, and regularization strategies, each tied directly to a lecture in the course. A separate **1D Convolutional Neural Network** experiment achieves 95% accuracy and demonstrates how CNN concepts from Lectures 5–7 transfer from image processing to text classification.

Beyond the notebooks, the project includes a fully deployed FastAPI backend and a React web frontend where a user can paste any news article and receive an instant prediction with a confidence score. This report documents every design decision, the experimental evidence behind it, and the practical engineering challenges encountered during implementation.

97.5%	912K	198	0
Test Accuracy (Main NN)	Trainable Parameters	Labeled News Articles	False Positives

Key results at a glance

1. Introduction

1.1 Problem Statement

Within the span of a few hours, a completely fabricated news story can be shared by hundreds of thousands of people across Facebook, Twitter, and WhatsApp. By the time professional fact-checkers publish a correction, the false version has already reached far more people — and studies consistently show that corrections rarely reach the same audience as the original misinformation. During elections, public health crises, and international conflicts, fake news has been shown to meaningfully change how people think, vote, and act.

The challenge from a technical perspective is that fake news does not have a fixed structure. It is written to look real — same tone, same grammar, same formatting as legitimate journalism. Sometimes it is entirely fabricated. Other times, it takes a real event and adds a misleading headline or removes critical context. A detection system cannot rely on formatting alone; it needs to understand the language patterns that distinguish fabricated content from genuine reporting.

1.2 Why Deep Learning?

Early automated approaches to fake news detection relied on handcrafted features — things like writing style metrics, source reputation scores, the number of uppercase words, and similar heuristics. While these features can help, they require domain experts to manually decide what matters, and they fail whenever misinformation writers adapt their style. Deep learning changes this fundamentally: instead of telling the model what to look for, the model learns what matters from labeled data.

For this project, the choice of deep learning was both practical and academic. Practically, neural networks can learn rich text representations without requiring extensive manual feature engineering. Academically, this task gave me the opportunity to apply every major concept from CSE 589 — perceptrons, activation functions, backpropagation, optimizers, regularization, and convolutional networks — to a single coherent real-world problem, rather than treating them as isolated theoretical exercises.

1.3 Project Objectives

#	Objective	Lecture Coverage
1	Build and train a Feedforward Neural Network for binary text classification	Lectures 2, 3, 4
2	Compare activation functions, optimizers, and regularization experimentally	Lectures 3, 4, 5
3	Implement a 1D Convolutional Neural Network and compare it against the main model	Lectures 5, 6, 7
4	Deploy the trained model as a live REST API connected to a React frontend	Practical
5	Justify every architectural decision with real experimental results	All lectures

2. Background

Fake news detection as a research area has progressed through several generations of approaches. Understanding this progression helps explain why a deep learning approach is appropriate and what its advantages are over simpler alternatives.

Rule-based systems were the earliest approach. These systems flagged articles based on explicit rules — articles from blacklisted domains, articles with excessive punctuation or all-caps words, or articles containing specific known false claims. They work reasonably well for detecting obvious spam but fail against well-crafted misinformation, because the rules cannot generalize beyond what was explicitly programmed.

Classical machine learning methods (Naive Bayes, SVM, Random Forest) improved on this by learning patterns from labeled data rather than hardcoded rules. TF-IDF features combined with logistic regression or SVM can achieve reasonable accuracy on curated datasets. The limitation is that these models cannot capture word meaning or sequence — they treat each document as an unordered bag of words, losing all context and phrasing information.

Deep learning approaches — feedforward networks on TF-IDF, CNNs on word embeddings, RNNs/LSTMs for sequential processing, and transformer models like BERT — represent the current state of the art. They learn abstract text representations automatically and can capture relationships between words that simpler methods miss entirely.

Why not BERT or transformers for this project? BERT-based models typically need tens of thousands of labeled examples and significant computational resources to fine-tune properly. With only 198 articles, fine-tuning BERT would almost certainly overfit — the model would memorize training examples rather than learning general patterns. More importantly, the goal of this course project is to demonstrate understanding of the lecture material from CSE 589, not to chase the highest possible benchmark number. A well-designed feedforward network and a 1D CNN serve that purpose far better.

3. Dataset

3.1 Data Collection and Labeling

The dataset contains 198 English-language news articles collected from various online sources and manually labeled as either Fake News (label = 1) or Factual News (label = 0). Articles were sourced from a mix of established news outlets and websites known to publish misinformation. Labeling was done by cross-referencing article claims against verified reporting from multiple reputable sources.

One of the first things that stands out when exploring the data is how balanced the class distribution is — 100 factual articles and 98 fake ones, nearly a perfect 50/50 split. This is genuinely convenient: the model cannot cheat by always predicting the majority class. A naive classifier that always outputs 'Fake' would be right only 49.5% of the time, so achieving 97.5% accuracy requires the model to have actually learned something about the text.

Property	Value	Notes
Total articles	198	No missing values in any row
Factual News (0)	100	50.5% — nearly perfectly balanced dataset
Fake News (1)	98	49.5% of the dataset
Original columns	4	text, title, label, source
After engineering	12	+8 features computed during preprocessing
Training samples	158	80% split, stratified to preserve class ratio
Test samples	40	Held out — never seen during training or validation

Table 1 — Dataset overview (Notebook 01)

3.2 Exploratory Analysis

Before training anything, I spent time understanding the data. Several patterns emerged. Fake news articles in this dataset tend to use more emotionally charged language and are slightly shorter on average than factual articles. Fake article titles more frequently use ALL CAPS words and excessive punctuation. Certain words appear almost exclusively in fake articles while others appear primarily in factual ones — which is precisely what TF-IDF is designed to capture.

The balanced class distribution meant no data augmentation or class weighting was needed. The 80/20 stratified split ensures that both training and test sets maintain the same class ratio, which is important for getting meaningful and interpretable accuracy numbers.

3.3 Engineered Features

Three additional numerical features were computed during preprocessing. These are not used as inputs to the neural network (which uses TF-IDF vectors), but they were useful for exploratory analysis and understanding what distinguishes the two classes:

Feature	Description	What It Revealed
text_length	Total character count of the full article	Fake articles were on average ~12% shorter
word_count	Tokens remaining after stopword removal	Closely correlated with text_length as expected
title_upper_ratio	Fraction of uppercase characters in the title	Notably higher in fake articles — clickbait pattern

Table 2 — Engineered features added during EDA (Notebook 01)

4. Methodology

4.1 Text Preprocessing Pipeline

Raw text cannot be fed directly into a neural network. The first transformation is cleaning — removing all the noise that would otherwise add meaningless dimensions to the feature space. I implemented a seven-step function that processes every article before it is vectorized. The same function is used both during training (in the notebook) and at inference time (in the API backend), which is essential for consistency:

Step	Operation	Rationale
1	Lowercase conversion	Prevents 'Election' and 'election' from being treated as separate words
2	URL and email removal (regex)	Web links and email addresses carry no semantic content for classification
3	HTML tag removal	Articles scraped from web pages sometimes contain leftover markup tags
4	Punctuation removal	Strips both ASCII punctuation and Unicode characters like smart quotes and em-dashes
5	Digit removal	Specific numbers (dates, statistics, IDs) rarely generalize across articles
6	Stopword removal	Removes 198 high-frequency English words (the, is, at...) that appear equally in all articles and therefore carry no discriminative information
7	Short-token filter	Drops any single-character tokens that survived the previous steps

Table 3 — Cleaning pipeline (backend/services.py and Notebook 01)

After this pipeline, each article becomes a clean string of 50 to 200 meaningful words. One practical lesson I learned: the cleaning function must be *identical* between training and the deployed API. An early version of the backend was using a slightly different cleaning order, which meant the vocabulary the model was trained on did not always match the tokens the API was feeding it. Unifying this into a single shared function (clean_text in services.py) fixed that problem.

4.2 Feature Extraction — TF-IDF

TF-IDF converts cleaned text into a fixed-length numerical vector that a neural network can process. The idea is straightforward: a word that appears frequently in a specific article (high Term Frequency) but rarely across the whole dataset (high Inverse Document Frequency) is likely meaningfully associated with that article's content and gets a high TF-IDF score. Common words that appear in almost every article get low IDF scores regardless of their frequency in any single document.

The vectorizer is configured with **max_features=5000** — keeping only the 5,000 most informative terms — and **min_df=2**, which discards words appearing in fewer than 2 articles (likely typos or rare proper nouns that cannot generalize). After fitting on the 158 training articles, the actual vocabulary contains **3,401 unique terms**. Each article is therefore represented as a 3,401-dimensional sparse vector. Before passing it to Keras, I call *.toarray().astype(np.float32)* to convert it to a dense format, since Keras Dense layers require dense float input.

> **Why TF-IDF and not word embeddings for the main model?** Word embeddings like Word2Vec or GloVe capture semantic relationships between words, but learning useful embeddings from scratch requires a much larger corpus. With only 198 training articles, random embedding initialization would barely converge into anything meaningful. TF-IDF is a statistical measure that works well on small datasets precisely because it does not need to learn word geometry — it just measures distinctiveness. The CNN experiment in Section 8 uses an Embedding layer, which lets us see the difference directly.

5. Neural Network Architecture

5.1 Layer Design and Reasoning

The main model is a Feedforward Neural Network (also called a Multilayer Perceptron) that takes the 3,401-dimensional TF-IDF vector and transforms it through three hidden layers before producing a single binary output. The architecture follows a funnel pattern — each layer compresses the representation further, forcing the network to learn increasingly abstract features:

Layer	Size	Activation	Params	Role
Input	3,401	—	—	One value per TF-IDF vocabulary term
Dense 1	256	ReLU	870,912	Heavy compression — learns high-level word combinations
Dropout 1	256	—	0	Randomly disables 50% of neurons per batch
Dense 2	128	ReLU	32,896	Further abstraction of learned features
Dropout 2	128	—	0	50% dropout continues the regularization
Dense 3	64	ReLU	8,256	Final feature compression before output
Dropout 3	64	—	0	30% dropout — lighter here, near the output
Output	1	Sigmoid	65	P(Fake News) — a probability value in (0, 1)
TOTAL	—	—	912,129	Total trainable parameters

Table 4 — Full model architecture (Notebook 02, model.summary())

Why ReLU in hidden layers? Sigmoid and Tanh both suffer from the vanishing gradient problem — their derivatives approach zero at the extremes, which means during backpropagation, earlier layers receive vanishingly small gradients and stop learning. ReLU's derivative is exactly 1 for any positive input, which keeps gradients healthy throughout the whole network. The experiment in Section 6.2 confirms this: Sigmoid needed 66 epochs to converge on this exact task, while ReLU needed only 23.

Why Sigmoid at the output? For binary classification, a single Sigmoid output is the natural choice. It maps any real number to a value between 0 and 1, which can be interpreted as the probability that the article is fake. A threshold of 0.5 gives the final class label. This pairs directly with Binary Cross-Entropy loss.

Why the 256 → 128 → 64 funnel shape? The input is very high-dimensional (3,401) relative to the dataset size (158 training samples). The first Dense layer aggressively compresses this into 256 features. Each subsequent layer halves the representation, allowing the network to progressively distill the most important patterns. Too gradual a compression would keep too many redundant features; too sudden a compression might destroy useful information.

5.2 Training Configuration

Choosing the right architecture is only part of the work. How the model is trained matters equally. Each hyperparameter below was chosen with a specific reason, and the relevant experiments are in Section 6:

Hyperparameter	Value	Reasoning
Loss function	Binary Cross-Entropy	The standard loss for a Sigmoid output (Lecture 4). Heavily penalizes confident wrong predictions, which encourages calibrated probabilities.
Optimizer	Adam (lr = 0.001)	Maintains per-parameter adaptive learning rates and uses momentum. Experiment 3 shows SGD at the same lr achieves only 60% accuracy.
Batch size	16	Mini-batch gradient descent. With 158 training samples, larger batches would make each epoch update less informative.
Max epochs	200	An upper ceiling only. Early Stopping always terminates training long before this limit is reached.
Early Stopping	patience=15, monitor val_loss	Halts training when validation loss stops improving and restores the best weights (Lecture 5). Primary mechanism against overfitting.
Validation split	15% (~24 samples)	Held back from the 158 training samples to monitor generalization. The 40 test samples remain completely unseen until final evaluation.
Train/Test split	80% / 20%, stratified	Stratified ensures both sets maintain the 50/50 class ratio.

Table 5 — Training configuration (Notebook 02)

6. Experiments

Notebook 03 runs four controlled experiments, each isolating one variable while keeping everything else constant. This is the only credible way to know whether a particular architectural choice actually makes a difference. A shared helper function (run_experiment) compiles, trains, and evaluates each model, returning accuracy, F1-score, and the number of epochs trained.

6.1 Experiment 1 — Perceptron Baseline (Lecture 2)

Before building the full multi-layer network, I tested the simplest possible model: a single-layer perceptron from Lecture 2. The architecture is just Input(3401) → Dense(1, Sigmoid) — no hidden layers, no Dropout. The result was **97.5% accuracy**, which honestly came as a surprise.

This result tells us something important about the data: TF-IDF features of these 198 articles are nearly linearly separable. A single hyperplane in 3,401-dimensional space can correctly classify most of them. The deeper architecture is not strictly necessary for *this* dataset, but it becomes essential when working with larger or more complex datasets where the decision boundary cannot be captured by a flat linear surface.

6.2 Experiment 2 — Activation Functions (Lecture 3)

Lecture 3 introduces the vanishing gradient problem and explains why modern networks use ReLU rather than Sigmoid or Tanh in hidden layers. I tested all three on the same 256 → 128 → 64 architecture with everything else held constant:

Activation	Epochs to Converge	Test Accuracy	Macro F1	What I Observed
Sigmoid	66	100.00%	1.0000	Eventually reached perfect accuracy but took nearly three times as long as ReLU. Saturating derivatives slow gradient flow through deep layers.
Tanh	20	100.00%	1.0000	Significantly faster than Sigmoid. Zero-centered outputs help, but the function still saturates at extreme values.
ReLU	23	97.50%	0.9750	Fast convergence, as expected. The slight accuracy gap vs Sigmoid may be due to some neurons becoming permanently inactive (dead ReLU).

Table 6 — Activation comparison on the same 256→128→64 architecture

Sigmoid's 66-epoch convergence vs ReLU's 23 is a direct empirical confirmation of the vanishing gradient theory from Lecture 3. On a simple dataset, Sigmoid can still reach 100% accuracy — but in practice with deeper networks or harder tasks, it would fail to converge at all. ReLU was chosen for the final model because efficiency of training matters, and the theory-practice alignment was exactly what the experiment was designed to demonstrate.

6.3 Experiment 3 — Optimizer Comparison (Lecture 4)

Lecture 4 explains that standard SGD with a fixed learning rate often struggles because the same step size is applied uniformly to all 912,000 parameters, regardless of their individual gradient magnitudes. Adaptive optimizers like Adam and RMSProp address this by maintaining per-parameter learning rates. The results were the starkest of any experiment in this project:

Optimizer	Learn Rate	Epochs	Test Accuracy	Analysis
SGD	0.01	16	60.00%	Just 10% above random guessing on a balanced binary dataset. The fixed LR causes erratic updates — overshooting in some parameter dimensions while barely moving in others.
RMSProp	0.001	23	97.50%	Adaptive scaling per parameter allows smooth navigation of the high-dimensional loss surface. Excellent performance.

Adam	0.001	23	97.50%	Combines RMSProp's adaptive rates with momentum. Same result as RMSProp here, but typically more robust across different tasks.

Table 7 — Optimizer comparison (Lecture 4)

The SGD result — 60% accuracy on a dataset where random guessing gives 50% — is the clearest demonstration in this entire project of why the course covers advanced optimizers. This is not a bug or a misconfiguration. It is the predicted behavior of a fixed-step optimizer on a high-dimensional non-convex loss surface with 912,000 parameters. Adam was selected for the final model because it consistently works well as a default across different architectures.

6.4 Experiment 4 — Regularization Comparison (Lecture 5)

A network with 912K parameters trained on 158 samples has the capacity to perfectly memorize the training set without learning anything that generalizes. Regularization is what prevents this. Lecture 5 covers two strategies: Dropout (randomly zeroing neurons during training) and L2 weight decay (penalizing large weights in the loss function). I tested both individually and combined:

Strategy	Setting	Epochs	Test Accuracy	Observation
No regularization	—	21	97.50%	Training accuracy reached ~100%. On a small dataset the model can memorize — but the test set is also simple enough that it still generalizes.
Dropout only	50% / 50% / 30%	22	97.50%	The chosen approach. Matches unregularized accuracy with similar training speed.
L2 only	lambda = 1e-4	157	97.50%	Same final accuracy but required 7x more epochs. The continuous weight penalty slows gradient updates significantly.
Dropout + L2	Combined	23	97.50%	No benefit beyond Dropout alone on this dataset. The L2 penalty adds computational cost without improving the result.

Table 8 — Regularization comparison (Lecture 5)

The most interesting finding here is not the accuracy — all strategies match — but the convergence speed. L2's 157 epochs vs Dropout's 22 is a 7x difference in training time for the same result. This is because L2 penalizes weights continuously, creating a constant "drag" on gradient updates. Dropout works differently: by randomly silencing neurons, it forces the network to develop redundant, distributed representations — which actually speeds up convergence when combined with Early Stopping, because the network cannot over-rely on any single neuron and must reach its solution more robustly.

7. Final Model Results

With the architecture and training configuration finalized — ReLU activations, Adam optimizer, three-layer Dropout (50%/50%/30%), and Early Stopping with patience=15 — the model is trained from scratch on 158 samples with a 15% validation split. Training stops at epoch 23 with the best weights restored from epoch 8. The model is then evaluated on the 40 held-out test samples that it has never seen during training or validation.

7.1 Classification Report

Class	Precision	Recall	F1-Score	Support	What This Means
Factual News (0)	0.95	1.00	0.98	20	All 20 factual articles correctly identified — not one false alarm
Fake News (1)	1.00	0.95	0.97	20	19 of 20 fake articles caught; one slipped through as 'Factual'
Macro Average	0.98	0.97	0.97	40	Balanced performance — neither class is systematically neglected

Table 9 — Classification report on the 40-sample test set (Notebook 02)

7.2 Confusion Matrix Analysis

	Predicted: Factual	Predicted: Fake
Actual: Factual	**TN = 20**	**FP = 0**
Actual: Fake	**FN = 1**	**TP = 19**

Table 10 — FP = 0 is the standout result. No real article was ever flagged as fake.

Zero False Positives (FP = 0) — The model never wrongly accused a genuine news article of being fake. In a real deployment, false positives are the most damaging error type: falsely labeling a legitimate source as misinformation could cause serious reputational harm and undermine trust in the system itself.

One False Negative (FN = 1) — One fake article was classified as real. This is the less severe error: the system missed one fake article, but it did not cause collateral damage to legitimate journalism. The model's confidence on this particular article was likely close to 0.5, meaning it was an ambiguous case.

Precision for Fake = 1.00 — Every article the model flagged as fake was genuinely fake. When the model raises an alarm, it is always correct.

Recall for Fake = 0.95 — The model caught 95% (19 out of 20) of all fake articles in the test set. The remaining 5% would require more training data or a lower classification threshold to capture, at the cost of some false positives.

8. 1D CNN for Text Classification (Lectures 5–7)

Lectures 5 through 7 covered Convolutional Neural Networks in the context of image recognition. The core idea is that small filters scan across an input, detecting local patterns regardless of where they appear — a vertical edge in an image, for instance. It turns out the exact same principle works for text, with one important difference: the filters are 1D and slide over sequences of word vectors instead of 2D pixel grids.

A 1D filter with kernel size 5 looks at 5 consecutive word vectors at a time and learns to detect specific 5-word phrase patterns that are predictive of the label. After sliding across the entire article, Max Pooling retains only the strongest activations — the most relevant patterns found anywhere in the text. The key insight from the lecture applies here too: weight sharing means we only need to learn one set of filter weights,

and they are applied everywhere in the sequence, making the representation efficient.

8.1 Why the Preprocessing Had to Change

TF-IDF is a bag-of-words representation — it throws away word order entirely. If you give it 'the government confirmed' and 'confirmed the government', you get identical vectors. Since a CNN's purpose is to detect local sequential patterns, word order must be preserved. The CNN therefore uses a completely different preprocessing pipeline:

Step 1: A Keras Tokenizer builds a vocabulary of the 5,000 most common words in the training corpus and assigns each word a unique integer ID.

Step 2: Each article is converted to a sequence of integer IDs representing its words in order.

Step 3: Sequences are padded to a fixed length of 200 tokens using pad_sequences() — shorter articles are zero-padded at the start; longer ones are truncated.

Step 4: An Embedding layer (inside the model) maps each integer ID to a 64-dimensional vector. These vectors are randomly initialized and learned during training — the model discovers what word representations work best.

8.2 Architecture

Layer	Output Shape	Params	What It Does
Embedding(5000, 64)	(200, 64)	320,000	Converts 200 word IDs into a 200×64 matrix of learnable word vectors
Conv1D(128 filters, k=5, ReLU)	(196, 128)	41,088	128 filters scan 5-word windows. Output length: (200-5)/1+1 = 196
MaxPooling1D(pool=2)	(98, 128)	0	Halves the sequence, keeping the strongest activation per 2-step window
Conv1D(64 filters, k=3, ReLU)	(96, 64)	24,640	Deeper 3-word patterns. Output length: (98-3)/1+1 = 96
GlobalMaxPooling1D	(64,)	0	Takes the global maximum — collapses any sequence length into a fixed 64-dim vector
Dense(64, ReLU)	(64,)	4,160	Classification head that maps extracted features to the output
Dropout(0.5)	(64,)	0	Regularization — prevents overfitting in the final layers
Dense(1, Sigmoid)	(1,)	65	Binary fake/real probability

Table 11 — 1D CNN architecture (Notebook 04)

8.3 Results and Architecture Comparison

Aspect	Feedforward Neural Network	1D CNN
Input representation	TF-IDF — bag of words, no order	Integer token sequences — word order preserved
How features are found	Global combinations via Dense layers	Local n-gram patterns via sliding Conv1D filters

Pooling	None	MaxPooling1D + GlobalMaxPooling1D
Trainable parameters	912,129	~390,000 (2.3x fewer)
Test accuracy	97.50%	95.00%
Epochs until stopping	23	45 — embedding layer takes more data to settle

Table 12 — Architecture comparison

The CNN achieves 95% accuracy with less than half the parameters of the Feedforward NN. The 2.5 percentage point gap is almost certainly a dataset size issue: the Embedding layer needs to learn 320,000 parameters to represent word meaning, and with only 158 training articles those embeddings barely converge into something useful. TF-IDF sidesteps this by using a direct statistical measure that works well on small corpora. With a dataset of several thousand articles, I would expect the CNN to match or exceed the Feedforward NN because it preserves information (word order, phrasing) that TF-IDF discards entirely.

9. System Architecture and Deployment

Training a model in a notebook is only useful if other people can use it. The full system consists of three components: the trained model files, a FastAPI backend that serves predictions through an HTTP API, and a React web frontend where users can interact with the system.

9.1 Backend — FastAPI

The backend (backend/main.py) is a FastAPI application that loads two files at startup: **nn_model.keras** (10.5 MB — the trained Keras model, saved with model.save()) and **vectorizer.pkl** (123 KB — the fitted TF-IDF vectorizer, saved with joblib.dump()). If either file is missing, the application logs an error and exits — it is not useful without both. Three HTTP endpoints are exposed:

Method	Endpoint	Example Response	Purpose
GET	/	{"message": "Fake News Detection API is running"}	Basic availability confirmation
GET	/health	{"status": "healthy"}	Returns 503 if the model failed to load
POST	/predict	{"prediction": "Fake News", "confidence": 0.8512}	Main inference endpoint

Table 13 — REST API endpoints (backend/main.py)

9.2 Prediction Pipeline

When the /predict endpoint receives a POST request with article text, it executes the following pipeline in sequence. Every step must match exactly what was done during training:

> **1. Input validation:** If the text is empty or contains fewer than 15 meaningful words after cleaning, the API returns a 400 error with an explanation. This was added after discovering that single words like 'great' would produce confident-looking but meaningless predictions.
>
> **2. Text cleaning:** The same clean_text() function from services.py is applied. Consistency between training and inference is critical — mismatched cleaning would cause the vectorizer to encounter unexpected token patterns.

3. TF-IDF vectorization: vectorizer.transform([cleaned]) converts the text to a 3,401-dimensional sparse vector using the vocabulary built during training.

4. Dense conversion: .toarray().astype(np.float32) — Keras Dense layers require a dense float32 NumPy array, not a scipy sparse matrix.

5. Model inference: model.predict(X, verbose=0) returns a float $p \in (0, 1)$.

6. Response construction: label = 'Fake News' if $p \geq 0.5$ else 'Factual News'. confidence = round(max(p, 1-p), 4). Both are returned as JSON.

9.3 Frontend — React

The frontend is a React 19 single-page application built with Vite and styled using Tailwind CSS 4. The interface is intentionally straightforward: a large text area for pasting an article, an Analyze button, and a results panel displaying the label and confidence score. Axios handles the POST request to the backend and extracts validation error messages from API responses, displaying them to the user in a readable format.

The backend is started from the backend folder using the virtual environment's uvicorn executable (.venv-dl\Scripts\uvicorn.exe main:app --host 0.0.0.0 --port 8000). The system Python (version 3.14) cannot be used because TensorFlow does not yet support Python 3.14.

10. Challenges Faced and How They Were Solved

A project of this scope rarely runs perfectly from start to finish. Several problems came up during development that required debugging and some independent problem-solving:

Python Version Conflict (TensorFlow / Python 3.14)

TensorFlow does not support Python 3.14, which is the latest version installed on the development machine. The first attempt to start the FastAPI server using the system Python failed immediately with ModuleNotFoundError: No module named 'tensorflow'. The fix required creating a dedicated virtual environment (.venv-dl) using Python 3.10, installing TensorFlow there, registering it as a Jupyter kernel, and configuring the uvicorn startup command to use that interpreter explicitly. This also means that any future dependency installation must target the virtual environment, not the system Python.

Model Format Mismatch (Random Forest vs Keras)

At one point during the project, the backend was loading a Random Forest model saved as model.pkl (scikit-learn format), but all the notebooks were training a Keras neural network. The server would start without errors but crash on any prediction request because the sklearn predict() API and the Keras predict() API have completely different signatures and input requirements. The fix was to update main.py to use tf.keras.models.load_model(), retrain and save the Keras model as nn_model.keras, and delete the old model.pkl.

Overconfident Predictions on Random Text

During testing, I discovered that typing a single word like 'great' into the frontend returned a confident 'Fake News' prediction at ~88%. This is technically correct behavior from the model's perspective: out of 3,401

vocabulary terms, one matched, and that term appeared more frequently in fake articles during training. But as a user experience it is clearly misleading and potentially dangerous. The fix was a minimum word count check in the API: if the cleaned text has fewer than 15 meaningful words, the request is rejected with a descriptive error explaining that the model requires a full article, not a short phrase.

Keras Model Naming Constraint

In the regularization experiment (Notebook 03), the combined Dropout + L2 model had its name set to 'Dropout_+_L2'. Keras model names cannot contain the '+' character. The training call crashed with an error about an invalid model name. The fix was simple once identified: sanitize the label before using it as a model name using .replace('+', 'plus') — but finding the root cause required reading through a fairly cryptic error message.

11. Conclusion

Looking back at what started as a course project and turned into a working end-to-end system, the thing I am most satisfied with is not the accuracy number itself — it is the fact that every design choice in this project is backed by an experiment I ran on real data. That is a more honest approach to machine learning than copying a configuration from a tutorial.

The main model achieves **97.5% accuracy with zero false positives** on the 40-sample test set. The controlled experiments in Notebook 03 turned course theory into concrete evidence. ReLU's convergence advantage over Sigmoid (23 vs. 66 epochs) is no longer just a slide from Lecture 3 — it is something I watched happen on my own training loss curves. Adam's superiority over SGD (97.5% vs. 60%) is not just a theoretical claim — it is a result I can point to with a table and a number.

The 1D CNN experiment demonstrated that convolutional architectures from Lectures 5–7 transfer directly to natural language processing. Achieving 95% accuracy with 2.3x fewer parameters than the Feedforward NN illustrates the efficiency of weight sharing. The accuracy gap is a predictable consequence of dataset size — not a flaw in the architecture — and would close with more data.

Building the deployment layer — the FastAPI backend, the prediction pipeline, the React frontend — added a dimension to the project that notebooks alone cannot provide. Debugging model loading failures, resolving Python version conflicts, and adding input validation are real software engineering challenges that any production ML system has to solve. Working through them gave me a much clearer picture of the gap between a trained model and something actually useful.

Future Directions

Larger dataset: With thousands of labeled articles, regularization differences would become more pronounced, and the 1D CNN would likely close the accuracy gap with the Feedforward NN because the Embedding layer would have sufficient data to learn meaningful word representations.

Pre-trained word embeddings: Initializing the CNN's Embedding layer with GloVe or Word2Vec vectors — trained on billions of words — would dramatically improve CNN performance on small datasets by giving the model a head start on understanding word semantics.

Recurrent architectures (LSTM / GRU): Long-range dependencies in articles — a claim made in the first paragraph that contradicts something in the last — are invisible to both TF-IDF and CNNs. Recurrent networks process sequences step-by-step and can theoretically capture these across-article patterns.

BERT fine-tuning: With a large enough dataset, fine-tuning a pre-trained transformer would represent the current state of the art. BERT understands context bidirectionally — it can distinguish 'bank' near 'river' from 'bank' near 'loan' automatically — a capability none of the models in this project possess.

References

[1] CSE 589 Neural Networks Lecture Notes, Lectures 2–7, Mustafa Kemal University, 2026.

[2] F. Rosenblatt, 'The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain,' Psychological Review, vol. 65, no. 6, pp. 386–408, 1958.

[3] D. P. Kingma and J. Ba, 'Adam: A Method for Stochastic Optimization,' International Conference on Learning Representations (ICLR), 2015.

[4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, 'Dropout: A Simple Way to Prevent Neural Networks from Overfitting,' Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014.

[5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, 'Gradient-Based Learning Applied to Document Recognition,' Proceedings of the IEEE, vol. 86, no. 11, 1998.

[6] A. L. Maas, R. E. Daly, P. T. Pham, et al., 'Learning Word Vectors for Sentiment Analysis,' Proceedings of ACL-HLT, pp. 142–150, 2011.

[7] TensorFlow / Keras Documentation, tensorflow.org/api_docs

[8] FastAPI Documentation, fastapi.tiangolo.com

[9] Scikit-learn Documentation — TfidfVectorizer, scikit-learn.org